43. Load Balancing and Reverse Proxy (Linux view)

43. Load Balancing and Reverse Proxy (Linux view)

Phase: Phase 6 — Networking
Estimated time: 1–2 days
Slug: `43-http-load-tls`

Learning objectives

- Explain reverse proxy role
- Read nginx/Caddy basics at config level
- Terminate TLS at proxy
- Health checks and upstreams

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Almost every production web service sits behind a reverse proxy or load balancer: nginx, HAProxy, Envoy, Caddy, or a cloud LB. On Linux you are the person who configures it, reads its logs, and explains why users see a 502. Understanding where TLS ends, who sees the real client IP, and how health checks remove broken backends is the difference between a five-minute fix and a long outage.

TLS termination means the proxy decrypts the client connection (decrypt-at- proxy). Traffic to the backend may then be plain HTTP on a private network, or re-encrypted. Knowing which you have decides where certificates live, what a packet capture shows, and which hop owns the expiry alarm.

Core concepts

Term	Meaning
Reverse proxy	accepts client requests, forwards to backends
Upstream / backend	the app servers behind the proxy
TLS termination	proxy decrypts client TLS (decrypt-at-proxy)
TLS passthrough	proxy forwards encrypted bytes, cannot read HTTP
Health check	proxy probes backends, removes failing ones
Balancing	round robin, least connections, hash
X-Forwarded-For	header carrying the original client address

```
  client --HTTPS--> [ proxy :443 ] --HTTP--> backend A :8080
                    |  cert+key     \--HTTP--> backend B :8080
                    |  decrypts here
                    +-- health check /healthz every 5s
          (TLS ends at the proxy; backend hop is clear unless re-encrypted)
```

502 Bad Gateway: proxy could not get a valid reply from the backend. 504 Gateway Timeout: backend answered too slowly. 503: no healthy backend or overload. Learn these three by heart.

Worked examples

Example 1: a minimal nginx reverse proxy with TLS termination.

```
upstream app {
    server 127.0.0.1:8080;
    server 127.0.0.1:8081;
}
server {
    listen 443 ssl;
    server_name app.example.com;
    ssl_certificate     /etc/ssl/app.crt;
    ssl_certificate_key /etc/ssl/app.key;
    location / {
        proxy_pass http://app;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Example 2: validate before reload, never reload blind.

```
sudo nginx -t
sudo systemctl reload nginx
```

Example 3: inspect what the client really receives.

```
curl -vk https://app.example.com/ 2>&1 | sed -n 1,25p
openssl s_client -connect app.example.com:443 -servername app.example.com </dev/
null 2>/dev/null | openssl x509 -noout -dates -subject
```

Example 4: tell proxy failure from backend failure.

```
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8080/healthz
curl -s -o /dev/null -w "%{http_code}\n" https://app.example.com/
sudo tail -n 20 /var/log/nginx/error.log
```

If the first is 200 and the second is 502, suspect the proxy to backend hop: wrong port, SELinux blocking the connect, or a firewall rule.

Example 5: check socket ownership.

```
sudo ss -tlnp | grep -E ':(80|443|8080)'
```

Mini-lab

1. In a lab VM install nginx and run a trivial backend: `python3 -m http.server 8080 --bind 127.0.0.1`.
2. Create a self-signed cert with `openssl req -x509 -newkey rsa:2048 -nodes -keyout app.key -out app.crt -days 7 -subj "/CN=lab.test"`.
3. Write a server block like Example 1 pointing at the backend. Run `nginx -t` and reload.
4. Request it with `curl -vk https://127.0.0.1/` and find the certificate subject and the HTTP status.
5. Stop the backend. Request again. Record the status code and the error log line. Restart the backend.
6. Say in one sentence where TLS ended and what an observer on the loopback backend hop could read.

Decision guide / failure modes

Symptom	Likely cause	First check
502	backend down, wrong port, connect blocked	`ss -tlnp`, error.log
504	backend slow, timeout too low	upstream latency, `proxy_read_timeout`
Cert warning	expired, wrong SAN, missing chain	`openssl s_client`, full chain file
Backend sees proxy IP	no forwarded header handling	`X-Forwarded-For` config
Redirect loop	backend thinks it is HTTP	pass `X-Forwarded-Proto`
One node gets all traffic	sticky hash, failed health check	upstream status, check config

Choose termination at the proxy when you need routing by path or host, caching, WAF, or central certificate management. Choose passthrough when end- to-end encryption is required and the proxy only needs SNI. Failure modes: forgetting `nginx -t`, private key world-readable, certificate renewed on disk but proxy never reloaded, and trusting `X-Forwarded-For` from the internet.

Common mistakes

- Single upstream without health checks
- Weak TLS configs
- Logging secrets at info level

Exercises

Work in `exercises/43-http-load-tls/`. Read that folder's `README.md` fully before coding.

```
cd exercises/43-http-load-tls
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Open `exercise.sh` and implement `tls_termination_means` so it prints one hyphenated token that states what the proxy does to the client TLS session when it terminates TLS. Use the vocabulary from the Core concepts table: all lowercase, words joined by hyphens, no extra output. No network access is needed. After it passes, build the mini-lab proxy and confirm with `curl -vk` which hop is encrypted. In your notes, also write which of 502, 503, and 504 you would expect when the backend is stopped, and which log file proves it. That short note is part of the exercise even though the test does not check it.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/43-http-load-tls/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. What is TLS termination?
2. Why health checks?
3. Where find nginx error log?

Further reading

- https://nginx.org/en/docs/